

MACHINE LEARNING

CNN for Image Recognition



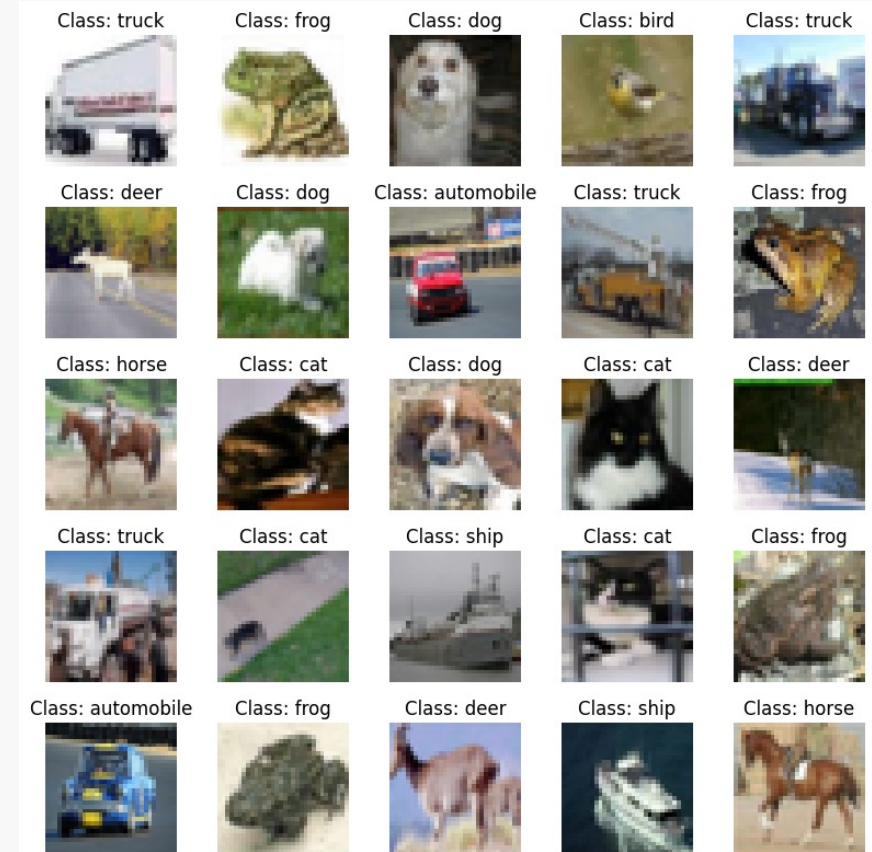
Agenda

- Data Set and Implementation Goal
- Architecture
- Activation and Loss Functions
- Data Partitioning
- Training Process
- Validation Strategy
- Testing and Results
- Critical Analysis
- Insights and Reflections

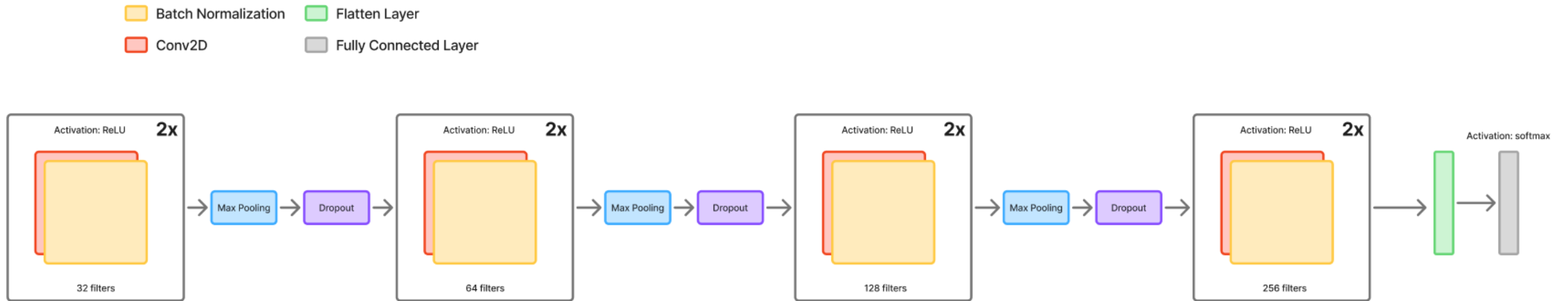


Data Set and Implementation Goal

- Neural Network-based Image classifier
- Train CNN with CIFAR-10 to classify images into 10 different classes



Architecture (VGG16 variant)



1,184,426 trainable parameters



Activation and Loss Functions

Activation Functions:

- ReLU (Conv2D layers)
- Softmax (Fully Connected Layer)

Loss Function: Categorical Cross Entropy

```
# Add the first convolutional layer with 32 filters of size 3x3
model.add(Conv2D(filters=32, kernel_size=(3,3), padding='same', activation='relu',
                 input_shape=X_train.shape[1:]))
# Add batch normalization layer
model.add(BatchNormalization())
```

```
# Add a fully connected layer with softmax
model.add(Dense(10, activation='softmax'))
```

```
# Compile the model with the defined optimizer, loss function, and metrics
model.compile(optimizer=optimizer, loss='categorical_crossentropy', metrics=['accuracy'])
```



Data Partitioning

```
X_train, X_valid, y_train, y_valid = train_test_split(X_train, y_train, test_size=0.15, random_state=0)
```

✓ 0.0s

Python

```
print('Train Images Shape:      ', X_train.shape)
print('Train Labels Shape:      ', y_train.shape)

print('\nValidation Images Shape: ', X_valid.shape)
print('Validation Labels Shape: ', y_valid.shape)

print('\nTest Images Shape:        ', X_test.shape)
print('Test Labels Shape:        ', y_test.shape)
```

✓ 0.0s

```
Train Images Shape:      (42500, 32, 32, 3)
Train Labels Shape:      (42500, 1)
```

```
Validation Images Shape: (7500, 32, 32, 3)
Validation Labels Shape: (7500, 1)
```

```
Test Images Shape:       (10000, 32, 32, 3)
Test Labels Shape:       (10000, 1)
```

Of the 60,000 CIFAR-10 Images:

- 42,500 were used for Training (70.8%)
- 7,500 were used for Validation (12.5%)
- 10,000 for Testing (16.7%)



Data Augmentation

```
# Data augmentation
data_generator = ImageDataGenerator(
    # Rotate images randomly by up to 15 degrees
    rotation_range=15,
    # Randomly flip images horizontally
    horizontal_flip=True,
    # Zoom images in by up to 10%
    zoom_range=0.1,
    # Change brightness by up to 10%
    brightness_range=[0.9,1.1],
    # Shear intensity (shear angle in counter-clockwise direction in degrees)
    shear_range=10,
    # Channel shift intensity
    channel_shift_range=0.1,
)
```

✓ 0.0s

```
# Data augmentation
data_generator = ImageDataGenerator(
    # Rotate images randomly by up to 15 degrees
    rotation_range=15,
    # Randomly flip images horizontally
    horizontal_flip=True,
    # Shift images horizontally by up to 12% of their width
    width_shift_range=0.12,
    # Shift images vertically by up to 12% of their height
    height_shift_range=0.12,
    # Zoom images in by up to 10%
    zoom_range=0.1,
    # Change brightness by up to 10%
    brightness_range=[0.9,1.1],
    # Shear intensity (shear angle in counter-clockwise direction in degrees)
    shear_range=10,
    # Channel shift intensity
    channel_shift_range=0.1,
)
```

✓ 0.0s

3 scenarios were tested:

1. No Augmentation
2. Intermediate Augmentation
3. Same augmentation as Nekouei, 2023.



Training Process

```
# Set the batch size for the training
batch_size = 64

# Set the maximum number of epochs for the training
epochs = 300

# Define the optimizer (Adam)
optimizer = Adam(learning_rate=0.0005)

# Compile the model with the defined optimizer, loss function, and metrics
model.compile(optimizer=optimizer, loss='categorical_crossentropy', metrics=['accuracy'])

# Add ReduceLRonPlateau callback
# Here, the learning rate will be reduced by half (factor=0.5) if no improvement in validation loss is observed
reduce_lr = ReduceLRonPlateau(monitor='val_loss', factor=0.5, patience=10, min_lr=0.00001)

# Add EarlyStopping callback
# Here, training will be stopped if no improvement in validation loss is observed for 40 epochs.
# The 'restore_best_weights' parameter ensures that the model weights are reset to the values from the epoch
# with the best value of the monitored quantity (in this case, 'val_loss').
early_stopping = EarlyStopping(monitor='val_loss', patience=40, restore_best_weights=True, verbose=1)

# Fit the model on the training data, using the defined batch size and number of epochs
# The validation data is used to evaluate the model's performance during training
# The callbacks implemented are learning rate reduction when a plateau is reached in validation loss and
# stopping training early if no improvement is observed
model.fit(data_generator.flow(X_train, y_train, batch_size=batch_size, shuffle=True),
          epochs=epochs,
          validation_data=(X_valid, y_valid),
          callbacks=[reduce_lr, early_stopping],
          verbose=2)
```

- Batch sizes of 64, 128 and 256 tested
- 300 max epochs
- Shuffling added



Validation Strategy

python

```
early_stopping = EarlyStopping(  
    monitor='val_loss',      # Monitors validation loss  
    patience=40,             # Waits 40 epochs for improvement  
    restore_best_weights=True, # Keeps best model weights  
    verbose=1  
)
```

python

```
reduce_lr = ReduceLR0nPlateau(  
    monitor='val_loss',      # Also monitors validation loss  
    factor=0.5,              # Reduces learning rate by half  
    patience=10,             # Waits 10 epochs before reducing  
    min_lr=0.00001          # Won't go below this learning rate  
)
```

- Fixed Pre-Validation set: 15% of the Training set is used for validation
- Early Stopping
- Reduce Learning Rate On Plateau

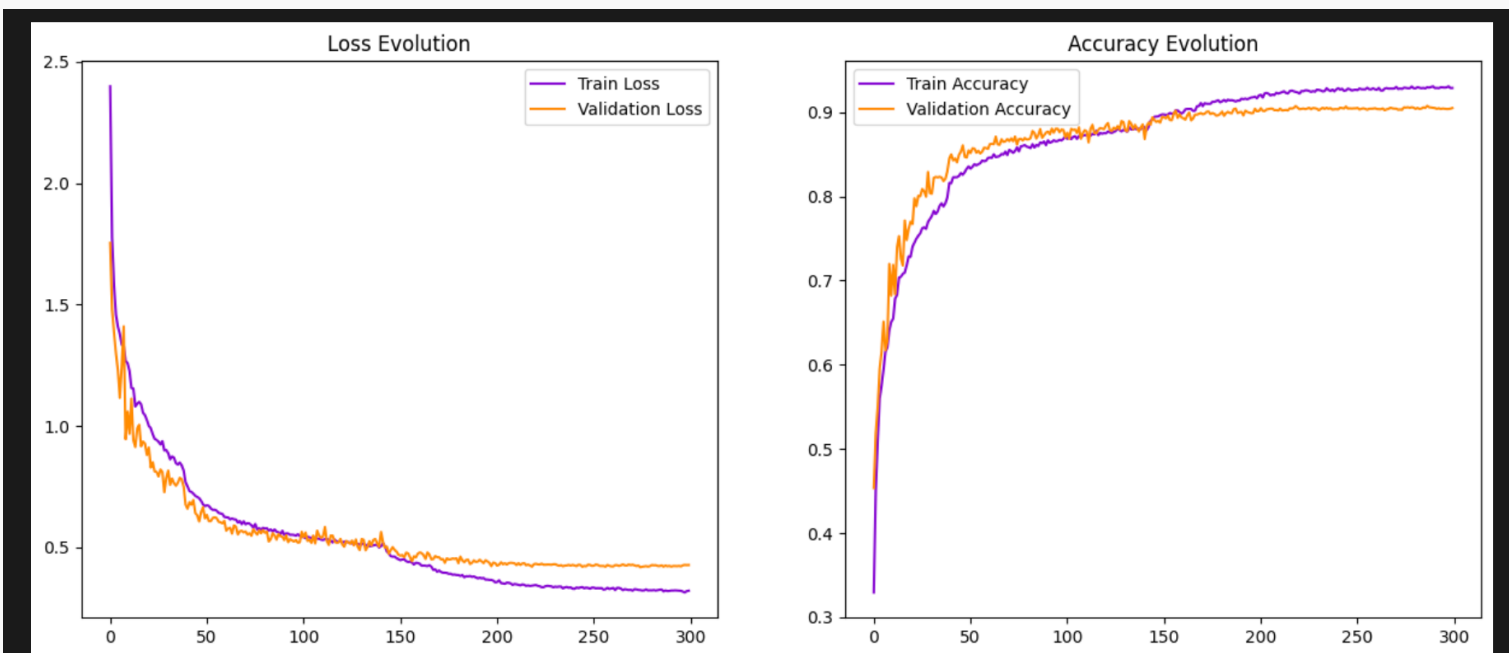


Tests Performed

Test #	Batch Size	Epochs	Avg Epoch Length	Data Augmentation	Accuracy (%)			Shuffling
					Training	Validation	Testing	
1	64	70	41s	None	97.96	87.34	85.47	No
2	256	282	16s	Medium	86.07	84.57	84.37	No
3	128	181	35s	Medium	90.23	87.65	87.06	No
4	64	192	37s	Medium	92.96	89.08	89.42	No
5	64	300	37s	High	92.91	90.57	90.11	No
6	64	207	38s	High	90.85	89.64	89.37	Yes



Tests Performed – Best Performer



- Batch Size of 64
- 300 epochs
- Highest Data Augmentation
- Accuracy higher than 90% across all 3 data sets

```
# Use the model to make predictions, evaluate on test data
test_loss, test_acc = model.evaluate(X_test, y_test, verbose=1)
```

```
print('\nTest Accuracy:', test_acc)
print('Test Loss:      ', test_loss)
```

✓ 3.1s

Python

313/313 ————— 3s 9ms/step – accuracy: 0.9012 – loss: 0.4509

Test Accuracy: 0.9010999798774719

Test Loss: 0.4514664113521576



Test – Image

```
# 1. First check the original image
print("Original image shape:", image.shape)

# 2. Resize the image to 32x32 (CIFAR-10 size)
image_resized = cv2.resize(image, (32, 32))
print("After resize shape:", image_resized.shape)

# 3. Convert to float32 and normalize
image_normalized = image_resized.astype('float32') / 255.0
print("After normalization shape:", image_normalized.shape)

# 4. Add batch dimension
image_batch = np.expand_dims(image_normalized, axis=0)
print("Final input shape:", image_batch.shape)

# 5. Now make prediction
prediction = model.predict(image_batch)

# 6. Print results
class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck']
predicted_class = class_names[np.argmax(prediction[0])]
print("\nPredicted class:", predicted_class)

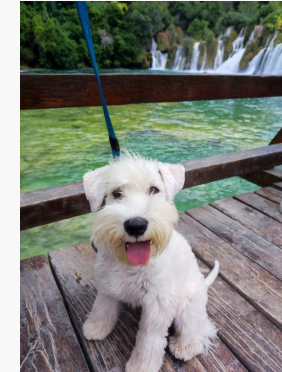
# 7. Print all probabilities
print("\nClass probabilities:")
for class_name, prob in zip(class_names, prediction[0]):
    print(f"{class_name}: {prob:.4f}")
```

✓ 0.0s Python

Original image shape: (256, 256, 3)
After resize shape: (32, 32, 3)
After normalization shape: (32, 32, 3)
Final input shape: (1, 32, 32, 3)
1/1 _____ 0s 47ms/step

Predicted class: frog

Class probabilities:
airplane: 0.0000
automobile: 0.0000
bird: 0.0049
cat: 0.0012
deer: 0.0021
dog: 0.0004
frog: 0.9912



- 2 dogs and 1 frog tested
- The model predicted the 3 as frogs



Critical Analysis

Challenges & Limitations:

- Epoch runtime
- Hard to individualise effects of single hyperparameters and data augmentation strategies
- Generalisation to real-world images

Future Improvements:

- New Data Augmentation Techniques
- Tensor slicing
- Use rented GPUs for Enhanced Speed
- Attempt implementation with CIFAR-100
- Try more complex architectures, like ResNet50

```
train_ds = tf.data.Dataset.from_tensor_slices((X_train, y_train))\
    .shuffle(1000)\
    .batch(batch_size)\
    .prefetch(tf.data.AUTOTUNE)

valid_ds = tf.data.Dataset.from_tensor_slices((X_valid, y_valid))\
    .batch(batch_size)\
    .prefetch(tf.data.AUTOTUNE)
```



Insights and Reflections

- Image Pipeline
- Overfitting
- Performance Considerations



Application to Real World Problems

Automatically Tag Images and create a mobile app that creates folders, enhancing the Photos App

